



UNIT 4 AI - Lecture notes

Artificial intelligence (Anna University)



Scan to open on Studocu

UNIT IV SOFTWARE AGENTS

Architecture for Intelligent Agents – Agent communication – Negotiation and Bargaining – Argumentation among Agents – Trust and Reputation in Multi-agent systems.

I. Intelligent Agent

Defn: is one that is capable of **flexible autonomous action** in order to meet its design objectives, where flexibility means three things

1. **reactivity:** able to perceive their environment(maintains an ongoing interaction) and respond to it in time to satisfy their design objectives
2. **pro-activeness:** able to exhibit goal-directed behavior by taking the initiative in order to satisfy their design objectives
3. **social ability:** are capable of interacting(cooperation, coordination, and negotiation) with other agents (and possibly humans) in order to satisfy their design objectives

Difference between agents and objects : three distinctions are

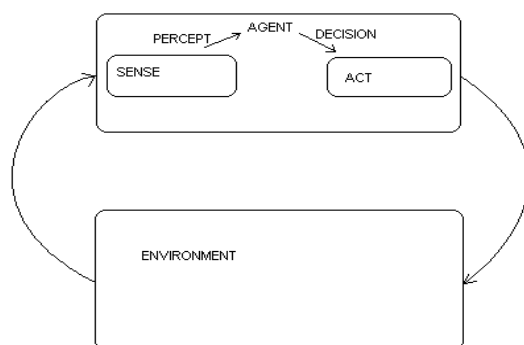
- a. agents are more autonomy than objects(they decide for themselves whether or not to perform an action on request from another agent)
- b. agents are capable of flexible (reactive, pro-active, social) behavior, and the object has nothing to say about such types of behavior
- c. a multiagent system is inherently multi-threaded, in that each agent is assumed to have at least one thread of control.

II. Architecture for Intelligent Agent

2.1 Abstract Architectures for Intelligent Agents

1. Standard agent

- assume the state of the agent's as a set $S = \{s_1, s_2, \dots\}$. At any given instant, the environment is assumed to be in one of these states.
- Let a set $A = \{a_1, a_2, \dots\}$ of actions.
- Then abstractly, an agent can be viewed as **Action: $S^* \rightarrow A$**



- the interaction of agent and environment is called as a **history**.
- History h is a sequence: $h: s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots s_u \xrightarrow{a_u}$

2. Purely Reactive Agents

- agents decide what to do without reference to their history, therefore they respond directly to their environment.
- the behavior of a purely reactive agent can be represented by a function

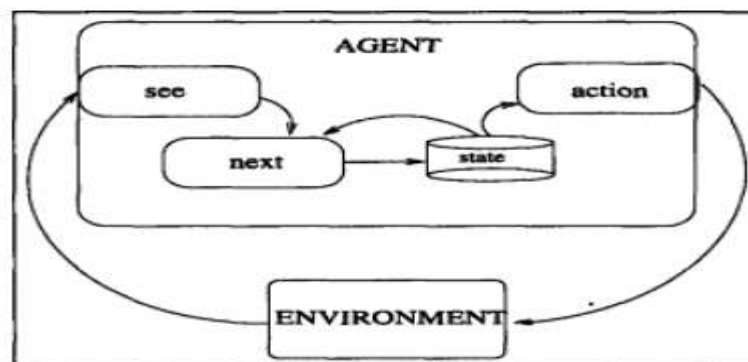
Action: $S \rightarrow A$

- e.g., thermostat agent
- ✓ goal is ***maintain room temperature***
- ✓ thermostat's environment can be in one of two states— ***either too cold or temperature OK.***
- ✓ therefore the thermostat's action function is

$$action(s) = \begin{cases} \text{heater off} & \text{if } s = \text{temperature OK} \\ \text{heater on} & \text{otherwise.} \end{cases}$$

3. Agent with state/ state-based agent

- Maintains an internal data structure=>contains information about the environment state and history.
- Let I be the set of all internal states of the agent
- An agent's do decision process based on the information I
- Pictorial Representation



- Functions used by the agent

1. see=> output of the **see function** is a **percept p**

See: $S \rightarrow P$

2. action-selection function => action is **maps internal states(sequence of percepts) to actions**

$action : I \rightarrow A$

3. next(next state function) => which maps an **internal state and percept to an internal state.** updates its view of the world when it gets a new percept.

$next : I \times Per \rightarrow I$

➤ **Agent Control Loop**

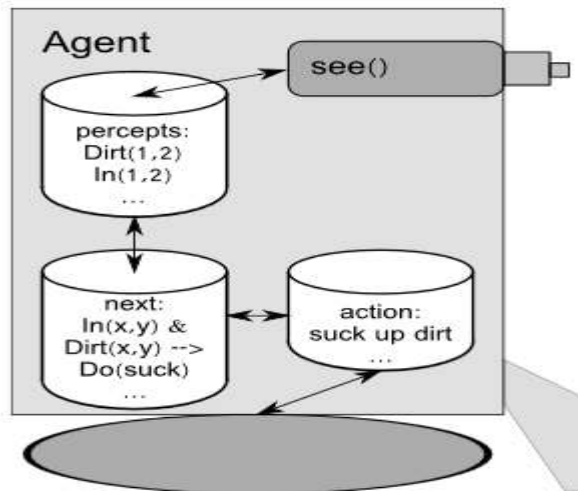
1. Agent starts in some initial internal state i_0 .
2. Observes its environment state e , and generates a percept **see(e)**.
3. Internal state of the agent is then updated via **next** function, becoming **next(i_0 , see(e))**.
4. The action selected by the agent is **action (next(i_0 , see(e)))**. This action is then performed.
5. Goto (2).

2.2 Concrete Architectures for Intelligent Agents

- In abstract architecture the abstract **function action**, indicates which action to perform—but the implementation of action is not shown/discussed
- four classes of agents
 1. Logic based agents
 2. Reactive agents
 3. Belief-desire-intention agents
 4. Layered architectures

1. Logic based agents

- Also called as **symbolic-based or deliberative architecture**
- agent behavior is based on the manipulation of the symbolic representation.
- logic-based architecture formalism is as follows:
 1. Assume that the environment is described by sentences in L and the knowledge base that contains all the information regarding the environment $KB = P(L)$ where $P(L)$ is the set of possible environments.
 2. For each moment of the time t , an agent's internal state is represented by $KB = \{KB_1, KB_2, KB_3 \dots KB_n\}$ where $KB_i \in KB$.
 3. The possible environment states are represented by $S = \{s_1, s_2, \dots\}$.
 4. An agent's reasoning mechanism is represented by a set of deduction rules, p which are the rules of inference.
 5. An agent perception functions as $see: S \rightarrow P$.
 6. The agent's internal state is updated using next: $KB \times P \rightarrow KB$.
 7. Thus, agent can choose an action from a set $A = \{a_1, a_2, \dots\}$, $action: KB \rightarrow A$ which is defined in terms of deduction rules. The outcome of an agent's actions is drawn via the **function do** where **do: $A \times S \rightarrow S$** .
- e.g., Vacuum cleaner
- A sketch of the agent



Possible precepts

PER= {dirt, null}

Domain predicates describing internal state

In(x)-> agent is at(x)

Dirt(x)-> there is dirt at(x)

Possible Actions

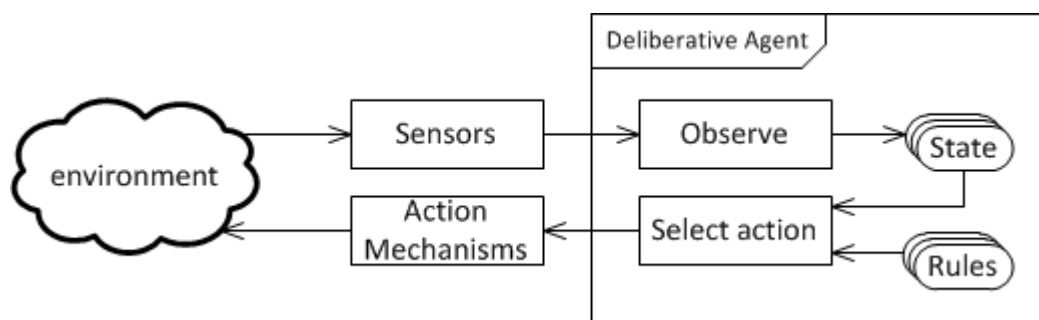
Actions={suck, ML, MR}

Deduction rules

$In(x) \wedge Dirt(x) \rightarrow Do(suck)$

$In(A) \wedge \neg Dirt(A) \rightarrow Do(MR)$

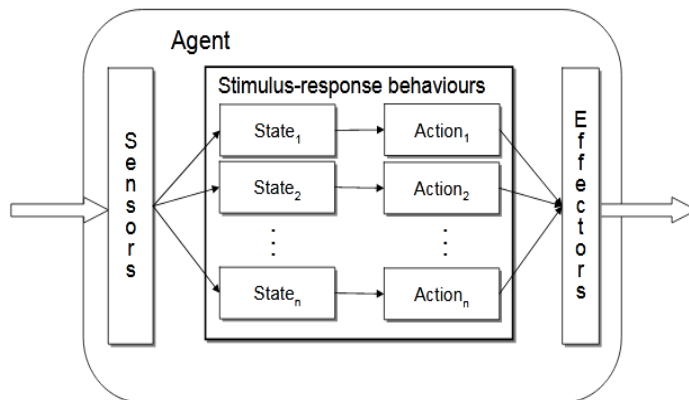
- disadvantages
 - Mapping real-world perceptions into logic statements is not ideal
- Pictorial representation



2. Reactive Architecture/ behavior-based model

- based on the **direct mapping of situation to action**
 - If <perceived situation> then <specifications>
- It is different from the logic-based architecture because **no symbols and complex symbolic reasoning are used.**

- The reactive architecture is realized through a set of sensors and effectors, where perceptual input is mapped to the effectors to changes in the environment.
- they do not take past events into account and cannot foresee the future and they do not revise their world (i.e., they do not maintain history)
- is very **flexible and adaptive** because they can manage their resource abilities in unpredictable worlds.
- Pictorial representation



3. **Belief-Desire-Intention (BDI) Architecture:**

- BDI works based on **Human practical reasoning**

Human practical reasoning consists of two activities:

1. Deliberation: deciding **what** to do i.e., form intentions.
2. Means-ends Reasoning: deciding **how** to do it i.e., form plans

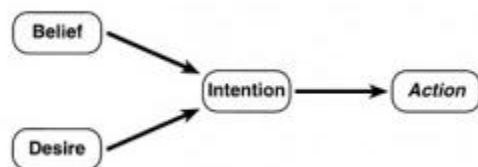
- **BDI** stands for: Belief, Desire, Intention
- Defn: BDI as whole can be represented by the following components

Belief => The agents **knowledge about world**

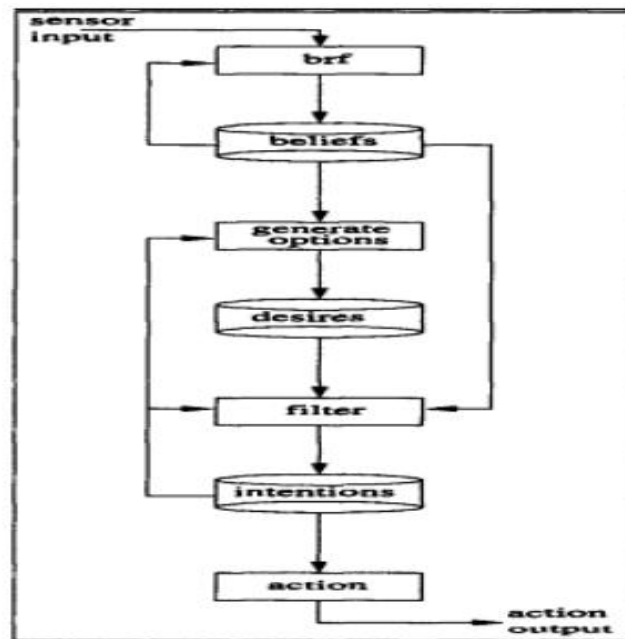
Desire => **possible goals** the agent can achieve based on the belief

Intention = the **course of actions** to achieve the desire of the agent/ they are plans that are currently being executed

- Pictorial representation of BDI



- Pictorial representation BDI process



Schematic diagram of a generic belief-desire-intention architecture.

.The process BDI agent

- ✓ a **belief revision function**, (brf), which takes a perceptual input and the agent's current beliefs=> determines a new set of beliefs;
- ✓ an **option generation function**, (options), which determines the options available to the agent (its desires), on the basis of its current beliefs about its environment and its current intentions
- ✓ a **filter function (filter)**, which represents the agent's deliberation(meaning is discussions) process, and which determines the agent's intentions on the basis of its current beliefs, desires, and intentions
- ✓ an **action selection function (execute)**, which determines an action to perform on the basis of current intentions.

Algorithm

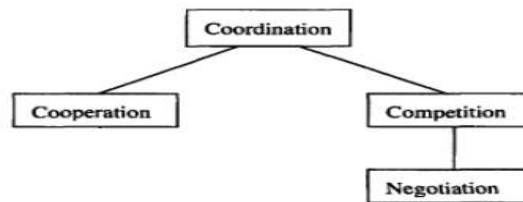
```

function action(p : P) : A
begin
  B := brf(B,p)
  D := options(D, I)
  I := filter(B, D, I)
  return execute(I)
end function action
  
```

III. Agent Communication

- Agents **have conversations**
- Agent communication is essential for **facilitation coordination, cooperation for agent interaction**

- Communication enables the agents to **coordinate their actions and behavior**, making agent to be coherent(meaning is rational)
- Taxonomy of different ways of agent can do coordination



- Coordination is a property of agents which **reduces resource contention, avoids livelock and deadlock, and maintaining applicable safety conditions.**
- **Cooperation** is coordination among **nonantagonistic (non hostile) agents**, while **negotiation** is coordination among **competitive or simply self-interested agents.**
- To cooperate successfully, each agent must maintain a model of the other agents, and also develop a model of future interactions
- Coherence is how well a system behaves as a unit.

3.1 Dimensions of Meaning

- Three important aspects of communications are
 1. Syntax (how the symbols of communication are structured)
 2. Semantics (what the symbols denote)
 3. Pragmatics (meaning is practical) (how the symbols are interpreted).

3.2 Message Types

- Agent **communicates via messages** which are categorized into
 1. Assertions
 2. Query
- **Structure of Agent message**

1. Message sender
2. Message receiver(s)
3. Language protocol
4. Encoding and decoding functions
5. Actions to be taken

3.3 Communication Levels

Semantics +Pragmatics Meaning(substance, type) of the information,
Syntax Format of information being transferred
Communication protocols Method of interconnection

3.4 Binary and n-ary communication protocols

- A binary protocol involves a single sender and a single receiver

- an n-ary protocol involves a single sender and multiple receivers (sometimes called broadcast or multicast).

3.5 Agent communication languages(ACL)

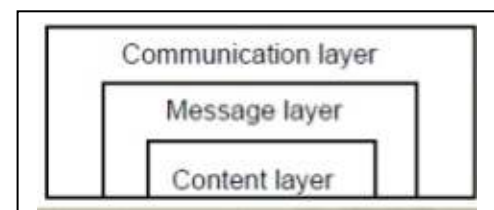
- A **language** that formalizes **how agents may interact** with one another and which is based on **Speech Act Theory**(is a theory of how utterances are used to achieve one's intentions)
- Speech act theory views human natural language as actions, such as requests, suggestions, commitments, and replies.
- speech act has three aspects
 1. Locution, the physical utterance by the speaker
 2. Illocution, the intended meaning of the utterance by the speaker
 3. Perlocution, the action that results from the locution.
- speech acts can be seen to have 2 components:
 1. A **performative verb** => e.g. Request, inform
 2. **Propositional content** => e.g. "the door is closed"

Speech Act	Please close the door	The door is closed	Is the door closed?
Performative	request	inform	inquire
Content	the door is closed	the door is closed	the door is closed

- Several types of speech act exist in human conversations is classified as
 1. Representatives (informing)=> which gives information. E.g., It is raining
 2. Directives(requesting) => which requires something from the recipient. E.g., pls make a tea
 3. Commissives(promising)=> which commit the sender to do something in the future. E.g., I promise to
 4. Expressive(thanking)=> which describe the mental states of the sender. E.g., thank you
 5. Declarations(declare)=> which process an act(showing emotions) just by saying it. E.g., declare war
- There are two main agent communication languages protocol

KQML(Knowledge query and manipulation language) protocol

- KQML is comprised of two parts:
 - the knowledge query and manipulation language (**KQML**)
 - the knowledge interchange format (**KIF**)
- is a **protocol** for **exchange information and knowledge**
- KQML is conceptually a **layered language**.
- Message format is divided into three layers
 1. Content layer



- ✓ Specifies the actual message written in agents known language
- ✓ KQML has different format of the original message content
- ✓ Therefore, the message content can be written in prolog code, C code, SQL, KIF

2. Message layer

- ✓ The core of KQML language
- ✓ Encodes a message that needs to be transferred.
- ✓ Primary function is to determine the network protocol used to deliver the message

3. Communication Layer

- ✓ Describes the **lower-level communication parameters**
- ✓ e.g., sender and receiver identity and unique identifier associated with the communication

– Message Structure of KQML

- A KQML message consists of performative name followed by parameter fields.
 - performative => a single word that tells about the message
 - Parameter name begins with colon(:)
 - ✓ The identity of the sender , receiver agent => communication layer
 - ✓ The language used in the content of the message
 - ✓ The ontology or vocabulary of the message=> within which the message content is to be interpreted
 - ✓ The message content
 - ✓ Example

```
(ask-one
  :sender      Gen-Agent
  :receiver    Gen-DB
  :reply-with  Q42
  :language    Prolog
  :ontology    Geneology
  :content     "mother('Liz',X)")
```

- ❖ Ask-one=> is the illocutionary uttered by speaker(Gen-agent) to the hearer(Gen-DB) for the response to the query given in message at :content.
- ❖ :language field indicates :content is expressed in prolog
- ❖ :ontology field informs the hearer " how to interpret the terms in the :content "
- ❖ :reply-with field carries a query identifier(q42) that shows the recipient should use this when answering (reply) back
- ❖ **Receivers reply => Example**

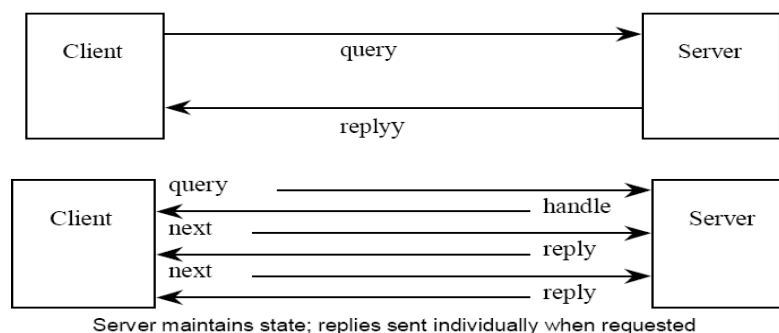
```
(tell
  :sender      Gen-DB
  :receiver    Gen-Agent
  :in-reply-to Q42
  :reply-with  Q43
  :language    Prolog
  :ontology    Geneology
  :content     "mother('Liz', 'Andy')")
```

- The language in a KQML is not restricted to KIF , other languages such as Prolog, Lisp and SQL
- KQML Performatives
 - o describes the types of the messages
 - o about 36 reserved performatives names
 - o performatives may be organized into seven basic categories

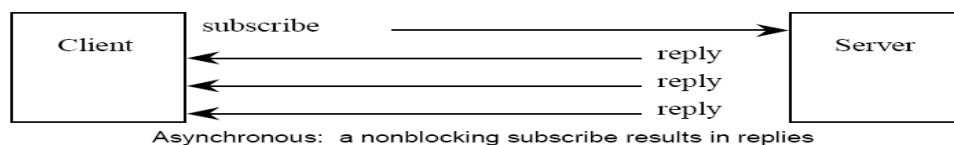
Category	Name
Basic query	evaluate, ask-if, ask-about, ask-one, ask-all
Multi-response (query)	stream-about, stream-all, eos
Response	reply, sorry
Generic informational	tell, achieve, cancel, untell, unachieve
Generator	standby, ready, next, rest, discard, generator
Capability-definition	advertise, subscribe, monitor, import, export
Networking	register, unregister, forward, broadcast, route,

- KQML speaking agents are treated as clients and servers.
- Communication can be either Synchronous or asynchronous

Synchronous => For a synchronous communication, a sending agent waits for a reply

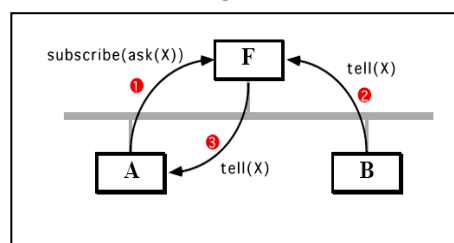


Asynchronous communication=> the sending agent continues its reasoning or acting which is interrupted when replies arrive at later time



Facilitators : an agent that performs communication services

- Maintain registry of service names
- Forward messages to named services
- Routing messages based on content (:ontology, :language etc)
- Provide matchmaking between information providers and requesters
- Provide mediation and translation services
- E.g., Agent A ask facilitator agent F to monitor for changes in its knowledge base



KIF(Knowledge Information Format)

- a language designed for the interchange of knowledge between agents
- KIF sentence are similar to first-order logic, expressed in prenex normal form (uses Lisp like syntax to represent first-order logic).
- KIF originated in a LISP application and inherits its syntax from LISP
- When a program reads a KB in KIF it converts the data into its internal form. process with this internal form , retransmits with KIF
- Two types

1. linear KIF

All expressions are strings of ASCII characters suitable for storage on serial devices (e.g., magnetic disks) and for transmission on serial media (as phone lines.)

2. Structured KIF

- All expressions are structured objects used in communication between programs
- In KIF word is taken as primitive.
- An expression can be either word or a finite sequence of the expression.
- In structured KIF parenthesis are used to bound the items in a composite expression.

`<word> ::= a primitive syntactic object`

`<expression> ::= <word> | (<expression>*)`

- The set of all words is divided into different categories
 - Variable : two types of variables
 - ✓ individual variables => begin with the ? Character
 - ✓ sequence variables => begin with an @ character.
 - ✓ `<indvar> ::= a word beginning with the character ?`
 - ✓ `<seqvar> ::= a word beginning with the character @`
 - Operators: used to form complex expressions and 4 types
 - ✓ term operators => `<termop> ::= listof | setof | quote | if | cond | the | setofall | kappa | lambda`
 - ✓ rule operators => `<sentop> ::= = | /= | not | and | or | => | <= | <=> | forall | exists`
 - ✓ sentence operators => `<ruleop> ::= ==> | <<= | consis`
 - ✓ definition operators => `<defop> ::= defobject | defunction | defrelation | := | :=> | :&`
 - Constant: a word neither variable nor operators and 4 types
 - object constants : denotes individual objects
 - function constants : denotes functions on those objects
 - relation constants : denotes relations

- logical constants : express conditions about the world, either true or false
- Format is
 - ✓ <objconst> ::= a word denoting an object
 - ✓ <funconst> ::= a word denoting a function
 - ✓ <relconst> ::= a word denoting a relation
 - ✓ <logconst> ::= a word denoting a truth value

Note:

- is not intended for interaction with humans
- is not intended to be internal representation for knowledge within computer programs
- is designed to serve as an mediator in the translation of other language

IV. Negotiation and Bargaining

4.1 Negotiation

- form of interaction that occurs among agents with different goals => negotiation
- Negotiation is a process by which a joint decision is reached by two or more agents, each trying to reach an individual goal or objective.
- The agents first communicate their positions, which might conflict, and then try to move towards agreement by making concessions or searching for alternatives.
- The major features of negotiation are
 - the language used by the participating agents
 - the protocol followed by the agents as they negotiate
 - the decision process that each agent uses to determine its positions, concession (compromise), and criteria for agreement.
- Techniques for Negotiation
 - a. Environment-centered
 - b. Agent centered
- a. **Environment-centered** : "How can the rules of the environment be designed so that the agents in it works fairly"
 - three types of environments identified: worth-oriented domains, state-oriented domains, and task-oriented domains
 - A task-oriented domain
 - ✓ agents have a set of tasks to achieve
 - ✓ all resources are available for the tasks
 - ✓ agents can achieve the tasks without help from each other.
 - However, the agents can benefit by sharing some of the tasks.
 - E.g., "Internet downloading domain"

Each agent is given a list of documents that it must access over the Internet. Cost is associated with downloading, which each agent would like to minimize. If a document is common to several agents, then they can save downloading cost by accessing the document once and then sharing it.

simple negotiation mechanism and constraints

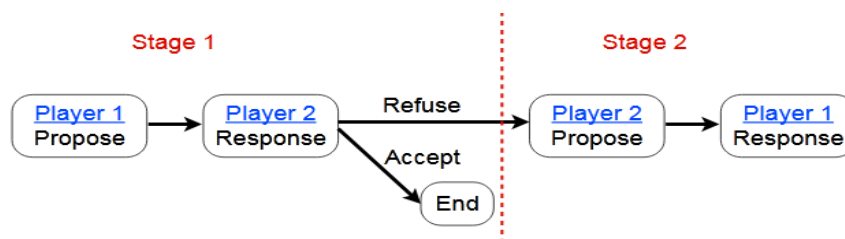
1. each agent declares the documents it wants
 2. documents common to two or more agents are assigned to agents based on the toss of a coin
 3. agents pay for the documents they download
 4. agents are granted access to the documents they download and shared documents
- This mechanism is simple, symmetric, distributed, and efficient (no document is downloaded twice).
- b. **Agent centered:** "Given an environment in which my agent must operate, what is the best strategy for it to follow?"

4.2 Bargaining

- agents can make a mutually beneficial agreement, but have a conflict of interest about which agreement to make
- Bargaining theory is a theoretic tool used to identify the bargaining solution, given
1. the set of all feasible agreements U
 2. the disagreement D
- **Subfields of Bargaining Theory**

1. Strategic Approach

- Modeling the bargaining process(i.e., the game form)explicitly
- Considering the game outcome(i.e., equilibrium)that results from the players' self-enforcing interactions.
- e.g., Rubinstein Bargaining Model,1982
- Pictorial representation of Bargaining solution by strategic approach



2. Axiomatic Approach

1. Abstracting away the details of the bargaining process
2. Considering only the set of outcomes that satisfy certain pre-defined properties (i.e., Axioms).
3. Typical Example: Nash Bargaining Model, 1950

Pre-Define Axioms

Axiom 1: Pareto efficiency
Axiom 2: Equal share of payoff gain
Axiom 3: Symmetry
...



Bargaining Solution(s):

The solution(s) that satisfies all axioms

- Nash analyzed a 2-agent setting where the agents have to decide on an outcome $o \in O$

- fallback outcome O_{fallback} occurs if no agreement is reached.

- There is a utility function $u_i : O \rightarrow \mathbb{R}$ for each agent $i \in [1, 2]$.

- It is assumed that the set of feasible utility vectors $\{(u_1(o), u_2(o)) | o \in O\}$ is convex

- Nash bargaining solution is the unique solution that satisfies the above 4 axioms.
 1. Pareto Efficiency
 2. Symmetry
 3. Invariant to Affine Transformations
 4. Independence of Irrelevant Alternatives

V. Argumentation among multi-agent

- Defn: Argumentation is a verbal and social activity of reason aimed at increasing (or decreasing) the acceptability of a controversial standpoint for the listener or reader, by putting forward a constellation of propositions intended to justify (or refute) the standpoint before a rational judge.
- Argumentation $\Rightarrow$ Reasoning about arguments $\Rightarrow$ decide on conclusion
- Dialectical argumentation = Multi-party argumentation through dialogue

1. Persuasion dialogues

- ✓ two or more participants try to resolve a conflict of opinion, each trying to persuade (meaning convince) the other participants to adopt their point of view
- ✓ E.g.,
 1. P : The newspapers have no right to publish information I.
 2. C : Why?
 3. P : Because it is about X's private life and X does not agree
 4. C : The information I is not private because X is a minister and all information concerning ministers is public
 5. P : But X is not a minister since he resigned last month

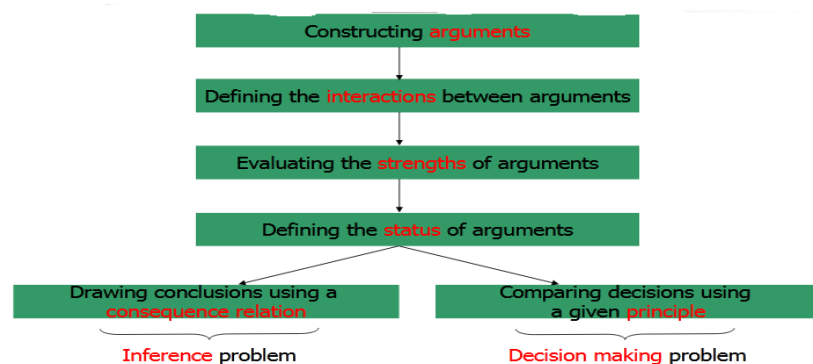
2. Negotiation Dialogue

✓ discussion between two parties that seeks to arrive at a destination of mutual agreement between the parties

✓ E.g.,

1. Buyer: Can't you give me this 806 a bit cheaper?
2. Seller: Sorry that's the best I can do. Why don't you go for a Polo instead?
3. Buyer: I have a big family and I need a big car (B1)
4. Seller: Modern Polo are becoming very spacious and would easily fit in a big family. (S1)
5. Buyer: I didn't know that, let's also look at Polo then.

➤ Argumentation Process



Types of arguments

1. **Explantations** (involve only beliefs)

Tweety flies because it is a bird

2. **Threats** (involve beliefs + goals)

You should do α otherwise I will do β

You should not do α otherwise I will do β

3. **Rewards** (involve beliefs + goals)

If you do α , I will do β

If you don't do α , I will do β

Interactions between arguments

1. **Rebutting attacks**: two arguments with contradictory conclusions

➤ E.g., Tweety flies because it is a bird versus Tweety does not fly because it is a penguin

2. **Assumption attacks**: an argument attacks an assumption of another argument

➤ E.g., Tweety flies because it is a bird, and it is not provable that Tweety is a penguin versus Tweety is a penguin

3. **Undercutting attacks**: An argument challenges the connection between the premisses and the conclusion

E.g., Tweety flies because all the birds I've seen fly

I 've seen Opus, it is a bird and it does not fly

- The **strength** of an argument depends on the quality of information used to build that argument
- **Status of arguments** : Some attacks can be removed



Three classes of arguments

1. Arguments with which a dispute can be won (justified)
2. Arguments with which a dispute can be lost (rejected)
3. Arguments that leave the dispute undecided

VI. Trust and Reputation in Multi-Agent Systems

- ✓ Trust => belief of another agent
 - ✓ Reputation=> opinion of the public towards an agent, based on past experiences
 - ✓ Reputation => measure to quantify trust
 - ✓ Models
1. **Interaction Trust** (reliability)=> agent's direct experience, takes long time
 - ✓ Agent A wants a service from agent B
 - ✓ Agent A judges if B is to be trusted from personal experience
 - ✓ Agent A trusts B and asks B to perform the service on A's behalf
 2. **Witness reputation** => reports of witness given by other agent's
 - ✓ Agent A wants a service from agent B
 - ✓ Agent A asks agent C if agent B can be trusted
 - ✓ Agent C trusts agent B and replies yes to A
 - ✓ Agent A now trusts B and asks B to perform the service on A's behalf

Implementation

1. Centralized approach

- One or more centralized trust authorities give trust estimations
- Convenient for witness reputation models
- Simpler to implement; better and faster trust estimations
- Less reliable

2. Decentralized (distributed) approach

- Each agent must estimate on its own the trust upon another agent
- Convenient for interaction trust models + Robustness: no single point of failure
- more realistic